

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	Y Greeshma Reddy	Reg. No.:	192424146
Section / Batch:	B Tech-AI&DS	Date:	24-07-2026

Instructions to Students

- Answer ALL debugging and optimisation tasks. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – DEBUGGING & OPTIMISATION TASK

Q1. Debugging Pair Sum Code (Same Element Allowed Through Incorrect Loop Design)

The following brute force Pair Sum implementation may incorrectly use the same element twice because of the way the inner loop is initialized and the missing restriction on pair formation. Carefully analyze why this leads to invalid solutions when the target equals twice the value of a single element. Debug the nested loops so that only distinct indices are considered and each valid unordered pair is checked exactly once. Optimize the implementation by avoiding redundant mirror comparisons. Analyze the time complexity of the corrected solution and rewrite the final version with proper comments.

```
def pair_sum(arr, target):  
    for i in range(len(arr)):  
        for j in range(i, len(arr)): # ERROR  
            if arr[i] + arr[j] == target:  
                return (i, j)  
  
    return None
```

Code of Conduct

I Y Greeshma Reddy (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y Greeshma Reddy

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (Each – 50)
Q1	4	3	3	2	4	39
Q2	4	4	4	3	2	44
Overall Total (100)						83

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Debugging Pair Sum Code (Same Element Allowed Through Incorrect Loop Design)

1. Problem Identification

The given Pair Sum program has two errors:

The inner loop starts from $j = i$, allowing the same element to be used twice (e.g., $\text{arr}[i] + \text{arr}[i]$).

The `return None` statement is incorrectly placed inside the outer loop, causing the function to stop after checking only the first element.

These issues may produce incorrect results and prevent all possible pairs from being checked.

2. Debugging

The errors are fixed by:

Changing the inner loop to `range(i + 1, len(arr))` so that only distinct indices are compared.

Moving `return None` outside both loops so the entire array is searched before concluding that no valid pair exists. This ensures that each valid unordered pair is checked exactly once.

Corrected code:

Pair Sum using Brute Force (Corrected)

```
def pair_sum(arr, target):  
    # Check all unique pairs  
    for i in range(len(arr)):  
        for j in range(i + 1, len(arr)): # Start from i+1 to avoid same element  
            if arr[i] + arr[j] == target:  
                return (i, j) # Return indices of the pair  
  
    return None # Return None if no pair is found
```

Driver Code

```
arr = [2, 7, 11, 15]
```

```
target = 9
```

```
result = pair_sum(arr, target)
```

```
if result:
```

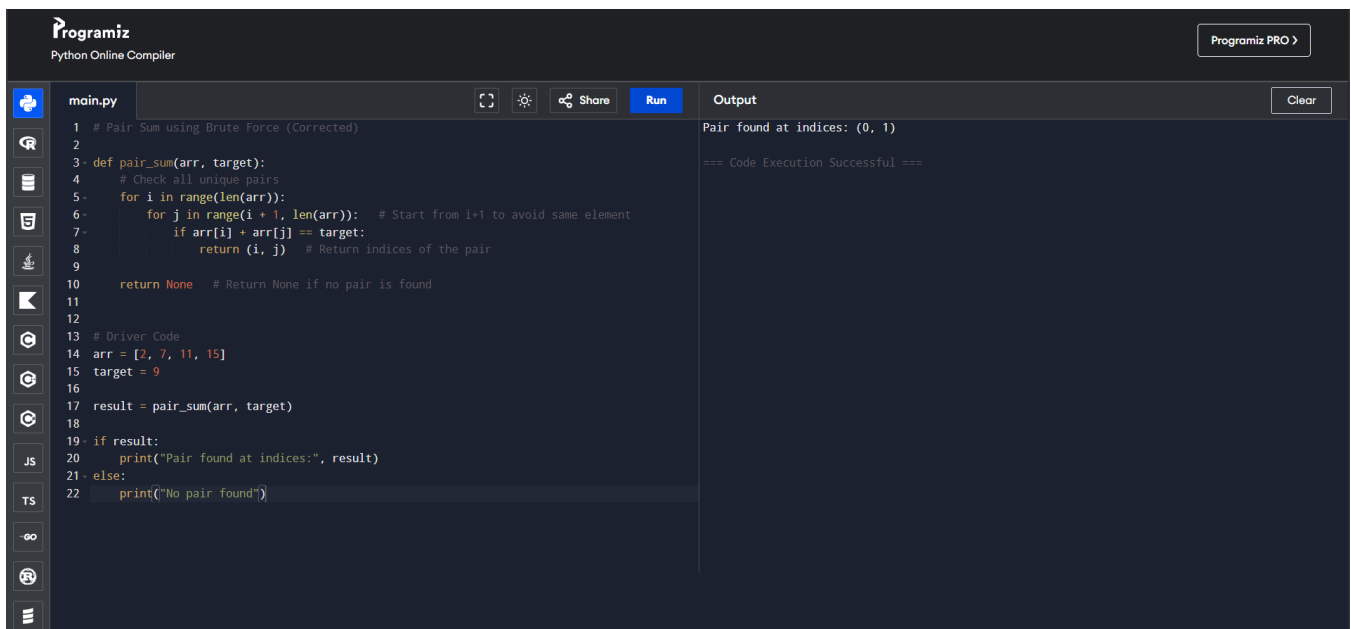
```
    print("Pair found at indices:", result)
```

```
else:
```

```
    print("No pair found")
```

3. Optimization

The corrected implementation avoids redundant comparisons such as (i, j) and (j, i) by starting the inner loop from i + 1. This reduces unnecessary checks while maintaining correctness. It also avoids comparing an element with itself.



The screenshot shows the Programiz Python Online Compiler interface. The editor contains a Python script named `main.py` with the following code:

```
1 # Pair Sum using Brute Force (Corrected)
2
3 def pair_sum(arr, target):
4     # Check all unique pairs
5     for i in range(len(arr)):
6         for j in range(i + 1, len(arr)): # Start from i+1 to avoid same element
7             if arr[i] + arr[j] == target:
8                 return (i, j) # Return indices of the pair
9
10    return None # Return None if no pair is found
11
12
13 # Driver Code
14 arr = [2, 7, 11, 15]
15 target = 9
16
17 result = pair_sum(arr, target)
18
19 if result:
20     print("Pair found at indices:", result)
21 else:
22     print("No pair found")
```

The output panel on the right shows the result of the code execution:

```
Pair found at indices: (0, 1)
=== Code Execution Successful ===
```

4. Analysis

Best Case: $O(1)$ – The required pair is found in the first comparison.

Worst Case: $O(n^2)$ – Every possible pair is checked before finding the answer or concluding no pair exists.

Space Complexity: $O(1)$ because only a few variables are used.

5. Code Quality

The corrected code is easy to understand, properly indented, well-commented, and follows good programming practices. It checks only valid pairs, avoids duplicate comparisons, and returns the correct output after examining all possible pairs.